TP Unidad 4 — Parte 1: Integración y despliegue continuos

Metodología de Sistemas II · Tecnicatura Universitaria en Programación · 2026

Entrega individual: Fabricio Ceniquel Thompson · Equipo Naft

Proyecto: Sistema Integral de Gestión — Centro Educativo «Educar para Transformar» (Resistencia, Chaco)

Repositorio: https://github.com/fabrithompson/CentroEducativo · **Producción:** https://transformar-para-educar.up.railway.app

Actividad 1 — Verdadero o falso

Contenido	V o F	Justificación
Flujo típico de Integración Continua. El desarrollador sube el cambio; el servidor de CI compila, corre pruebas unitarias y de integración y analiza la calidad; si todo pasa se puede fusionar, y si falla se notifica al equipo.	**V**	Es exactamente lo que hace nuestro CI en GitHub Actions: en cada push y en cada PR corre typecheck, lint, pruebas de integración contra PostgreSQL, pruebas de la app móvil y Selenium. El merge a `main` queda bloqueado si algo falla, y GitHub notifica por correo.
Proceso de Integración Continua. Ramas separadas que se integran con frecuencia en la principal; el servidor compila y prueba automáticamente; si hay error se notifica para corregirlo rápido.	**V**	Describe el flujo de ramas por funcionalidad con PR hacia `main`. La notificación temprana es lo que evita que un error se acumule con otros.
Principios clave de la CI. Integrar con frecuencia, mantener el código listo para desplegar, automatizar pruebas, usar control de versiones y revisar los resultados del CI antes de aprobar merges.	**V**	Son los cinco principios. El último lo implementamos como regla: la protección de `main` exige los seis chequeos en verde para mergear.
Relación con Entrega Continua (CD). La CI asegura que el código nuevo se integre bien; la Entrega Continua automatiza el despliegue en entornos de prueba o producción; juntas forman el flujo CI/CD.	**V**	Con una precisión: en la *entrega* continua el código queda siempre listo y el paso a producción puede requerir una aprobación manual; cuando también es automático se habla de *despliegue* continuo. Nuestro proyecto hace despliegue continuo: Railway publica solo cada merge a `main`.
Flujo típico de un pipeline CI/CD. Commit → Build → Test → Package → Deploy → Notify.	**V**	Coincide con nuestro pipeline: push → GitHub Actions (build con `tsc`, pruebas) → Railway construye la imagen (package) → aplica migraciones y despliega → GitHub y Railway notifican el resultado.
Etapas típicas del despliegue automatizado. Build, Test, Package, Deploy, Verify.	**V**	En nuestro caso, *Verify* hoy es parcial: si las migraciones fallan en el pre-deploy, Railway no publica y queda sirviendo la versión anterior, y la aplicación expone `GET /health`. Pero Railway todavía no está configurado para consultar ese endpoint antes de pasarle tráfico (`healthcheckPath`): es una mejora pendiente.

Actividad 2 — Beneficios de la Integración Continua

Beneficio	Descripción
Detección temprana de errores	Los fallos se detectan en minutos, no semanas después.
Reducción de conflictos de integración	Al integrar frecuentemente, se evitan «infiernos de merge».
Mayor calidad del software	Las pruebas automáticas garantizan estabilidad.
Entrega más rápida	El producto está siempre en condiciones de ser desplegado.
Transparencia	Todos pueden ver el estado del proyecto (builds, pruebas, cobertura).

Actividad 3 — Herramienta propuesta: Dependabot

Proponemos **Dependabot**, el servicio de GitHub que vigila las dependencias del proyecto y abre solo un *pull request* cuando hay una versión nueva o una vulnerabilidad publicada.

Por qué para este proyecto:

- El sistema maneja datos personales de menores, domicilios y teléfonos de los tutores. Una dependencia con una vulnerabilidad conocida (en `express`, `jsonwebtoken`, `multer` o `bcrypt`, por ejemplo) es un riesgo concreto, y hoy nadie revisa eso en forma sistemática. Al instalar, pnpm ya informa nueve subdependencias deprecadas.
- **Se integra con lo que ya está**: cada PR de Dependabot pasa por el mismo CI (typecheck, integración, E2E), así que una actualización que rompe algo se ve antes de mergear y no en producción.
- **Es gratis** para repositorios públicos y se configura con un solo archivo, `.github/dependabot.yml`: ecosistema `npm` (es compatible con pnpm), frecuencia semanal y agrupación de actualizaciones menores para no llenar el repositorio de PR.
- **Cierra el ciclo con la Unidad 3**: mantener las dependencias al día es una buena práctica que, sin automatizar, depende de la memoria de alguien.

Actividad 4 — Preguntas

Relación entre la Unidad 3 (buenas prácticas de programación) y la Unidad 4

La Unidad 3 dice *qué* es código de calidad: nombres claros, funciones chicas, manejo de errores, pruebas, control de versiones, revisión entre pares y convenciones compartidas. La Unidad 4 dice *cómo garantizar*

que se cumpla siempre, sin depender de que alguien se acuerde. Cada buena práctica tiene su control automático:

Buena práctica (Unidad 3)	**Cómo la hace cumplir el CI/CD (Unidad 4)**	**En este proyecto**
Convenciones de estilo y código limpio	Linter en el pipeline y antes de cada commit	ESLint en el editor, en Husky y en el CI
Pruebas automatizadas	El pipeline las corre en cada cambio	382 pruebas + 3 E2E en cada PR
Control de versiones y commits descriptivos	Hooks que validan el mensaje	`commit-msg` exige Conventional Commits
Revisión entre pares	PR obligatorio antes de integrar	Protección de `main` + `CODEOWNERS`
No romper lo que funciona	Pruebas de regresión antes del merge	Selenium verifica RF-03 en cada PR
Configuración fuera del código	Variables de entorno por entorno	`.env.example` en local; variables de Railway en producción

¿Cómo modifica CI/CD el proceso tradicional y qué beneficios aporta?

En el proceso tradicional, cada integrante trabaja aislado durante semanas, la integración es una fase al final, las pruebas se hacen a mano cuando hay tiempo y el despliegue es una tarea manual y riesgosa. CI/CD lo invierte: se integra varias veces por día, cada cambio se verifica solo y el despliegue pasa a ser un evento rutinario.

- **Calidad**: cada cambio pasa por los mismos controles, sin excepciones. En este proyecto, un despliegue anterior llegó a producción sin aplicar las migraciones porque dependía de que alguien las corriera a mano; hoy el pre-deploy de Railway las aplica siempre y, si fallan, no publica.
- **Tiempo**: el CI da una respuesta en menos de un minuto y medio. Del merge a la aplicación respondiendo en producción pasan entre 3 y 5 minutos, sin intervención humana.
- **Trazabilidad**: cada cambio en producción se puede seguir hacia atrás: despliegue de Railway → commit → PR → issue que lo pidió. Antes de este TP faltaba la última parte: había 0 issues y el 53 % de los commits llegaba a `main` sin PR.
- **Productividad**: el equipo deja de gastar tiempo en tareas repetitivas (compilar, probar, desplegar, respaldar la base) y lo dedica a funcionalidades. El respaldo diario de la base, por ejemplo, corre solo y además se verifica restaurándolo.

¿Qué mejoras introduce CI/CD en el trabajo colaborativo y en la calidad?

- **Una sola fuente de verdad**: lo que dice el CI es lo que vale para los dos integrantes. Se termina el «en mi máquina anda».
- **La revisión se enfoca en lo importante**: si el linter y las pruebas ya pasaron, la revisión humana puede concentrarse en la lógica y en las reglas de negocio, no en el formato.
- **Integración frecuente y en partes chicas**: los PR chicos se revisan rápido y generan menos conflictos. Una de las mediciones de este TP muestra lo contrario: el 29/09 se empujaron seis cambios seguidos directamente a `main` y los despliegues se encolaron hasta 34 minutos.
- **Visibilidad compartida**: el tablero Kanban, los issues y el estado del CI muestran a todos qué se está haciendo, qué está bloqueado y qué ya está en producción.

¿Cómo se podría incorporar la IA a los contenidos de la Unidad 4?

- **Revisión de PR asistida**: un modelo de IA comenta cada PR antes de la revisión humana. Detecta posibles defectos y verifica la lista de control de la plantilla.
- **Diagnóstico de fallas del pipeline**: ante un job en rojo, la IA lee el log y resume la causa probable. En este proyecto se usó así para descubrir que el build de Railway fallaba por el builder, no por el código.
- **Generación de pruebas**: proponer casos de prueba a partir de los criterios de aceptación de un issue, que una persona revisa antes de incorporarlos.
- **Análisis de métricas**: leer las métricas de entrega, como las de este TP, y señalar cuellos de botella y tendencias.
- **Infraestructura como código**: redactar y revisar archivos de pipeline y de infraestructura (`ci.yml`, `.railway/railway.ts`).

El equipo ya usa IA en el desarrollo y registra cada intervención en `docs/bitacora_ia.md`, como exige la materia. La condición para que sume y no reste es la misma que para cualquier otro aporte: **nada de lo que propone la IA llega a `main` sin pasar por el CI y por la revisión de una persona**.